# Scheduling Mechanism & Policy

2026 Semester 2 COMPSCI 340: Operating Systems
Talia Xu

Lecture 7
2.1.1

Based on original slides by Professor Jon Eyolfson.

# Scheduling Connects CPU Control to a Policy Choice

A ready process can run only after the OS gives it a CPU.

This lecture follows one complete scheduling cycle:

1. A process executes directly on the CPU
2. A system call, exception, or interrupt returns control to the kernel
3. The scheduler decides whether another process should run
4. A context switch restores the selected process

We then compare policies using the same set of CPU bursts.

## Direct Execution Lets the Process Use the CPU Efficiently

When a process is running, its instructions execute directly on the CPU.

The kernel is not interpreting every instruction. It becomes active when:

- the process deliberately requests a service,
- the current instruction causes an exception, or
- hardware generates an interrupt.

A switch can begin only after one of these events returns control to the kernel.

## A Function Call Stays Inside the Current Process

For an ordinary function call:

- the CPU remains in user mode,
- arguments follow the language calling convention,
- the return address and local state use the user stack, and
- execution jumps to code already available to the process.

The process changes functions, but it does not give the kernel control.

# A System Call Requests Work That Requires Kernel Privilege

Calls such as `write`, `fork`, and `pipe` eventually request protected kernel operations.

Unlike an ordinary function call, a system call must:

- change from user mode to kernel mode,
- enter at a kernel-approved address,
- preserve the user process's CPU state, and
- use memory the kernel trusts.

## Returning from a Trap Can Resume the Same Process or Another One

Before leaving the kernel, the scheduler may keep the current process or select a different ready process.

Return-from-trap then:

- restores the selected process's saved registers,
- restores its program counter and stack pointer,
- returns to user mode, and
- continues at that process's next user instruction.

A trap entry creates the opportunity to schedule; it does not require a context switch.

## Blocking and Yielding Give the Kernel Cooperative Control

A process cooperates when it enters the kernel and cannot or does not need to keep the CPU.

Examples include:

- a `read` that must wait for input,
- `wait` while a child is still running,
- an explicit yield, or
- termination.

Cooperative events are useful, but a CPU-bound process may avoid them for an arbitrarily long time.

## Timer Interrupts Let the Kernel Preempt CPU-Bound Code

Consider a process that never makes a system call:

```
for (;;)
    work++;
```

A timer device interrupts periodically. The kernel:

1. handles the timer event,
2. accounts for the process's CPU time, and
3. may preempt it so another ready process can run.

# A Scheduler Combines a Choice with a Context Switch

**Policy**

- Select the next ready process
- Set how long it may run
- Prioritize the desired metric

**Mechanism**

- Save the current CPU state
- Select another saved state
- Restore it safely

Preemptive scheduling uses timer interrupts to apply the policy even when the current process could continue.

# A Context Switch Moves the CPU from Process A to Process B

1. A trap or interrupt enters the kernel while A is running
2. The kernel saves A's registers and updates A's process state
3. The policy selects ready process B
4. The kernel switches to B's address-space and kernel-stack state
5. It restores B's saved registers
6. Return-from-trap resumes B in user mode

The kernel never copies "the CPU" itself; it saves and restores the state that defines each execution.

## Scheduling Policies Choose Among Ready CPU Bursts

A **CPU burst** is one continuous period of CPU execution between blocking events.

For the first policy examples, assume:

- one CPU,
- known arrival and burst times,
- no I/O during a burst, and
- zero context-switch cost.

These assumptions isolate the policy decision before later lectures add realistic complications.

## Different Metrics Describe Different Scheduling Goals

- **Waiting time:** total time ready but not running
- **Response time:** first run time minus arrival time
- **Turnaround time:** completion time minus arrival time
- **Throughput:** jobs completed per unit of time
- **CPU utilization:** fraction of time the CPU does useful work
- **Fairness:** whether runnable jobs receive a reasonable share

Improving one metric can make another worse.

## FCFS Runs Ready Processes in Arrival Order

First Come First Served keeps a FIFO ready queue.

- The oldest ready process runs next
- The running process continues until it finishes or blocks
- The policy is simple and has little scheduling overhead
- A long job at the front can delay every short job behind it

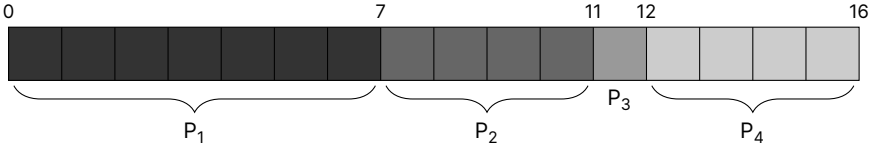
That last effect is called the **convoy effect**.

# Trace FCFS with Four Ready Processes

Consider the following processes:

| Process | Arrival Time | Burst Time |
|---|---|---|
| $P_1$ | 0 | 7 |
| $P_2$ | 0 | 4 |
| $P_3$ | 0 | 1 |
| $P_4$ | 0 | 4 |

Assume, $P_1 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4$. For FCFS, our schedule is:

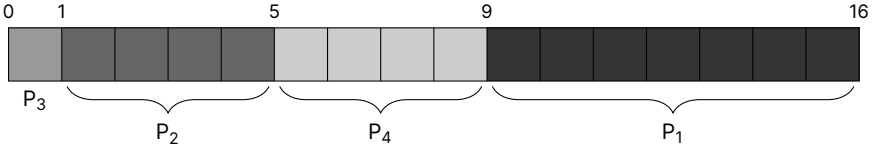


# Trace FCFS When the Short Job Goes First

Consider the same processes:

| Process | Arrival Time | Burst Time |
|---|---|---|
| $P_1$ | 0 | 7 |
| $P_2$ | 0 | 4 |
| $P_3$ | 0 | 1 |
| $P_4$ | 0 | 4 |

Assume, $P_3 \rightarrow P_2 \rightarrow P_4 \rightarrow P_1$. For FCFS, our schedule is:



## FCFS Waiting Time Depends Strongly on Arrival Order

| **Order** | Waiting times | Average |
|---|---|---|
| $P_1, P_2, P_3, P_4$ | $0, 7, 11, 12$ | 7.5 |
| $P_3, P_2, P_4, P_1$ | $0, 1, 5, 9$ | 3.75 |

The CPU performs the same 16 units of work in both schedules. Only the order changes, but moving short jobs forward halves the average wait.

## SJF Runs the Shortest Ready Burst First

Shortest Job First chooses the ready process with the smallest predicted CPU burst.

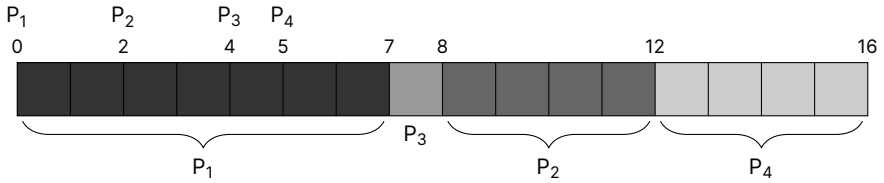
- It is non-preemptive: a chosen burst runs until it finishes or blocks
- Short work avoids waiting behind long work
- If only one process is ready, it must run even if shorter jobs arrive later

# Trace SJF with Staggered Arrivals

Consider the same processes with different arrival times:

| Process | Arrival Time | Burst Time |
|---|---|---|
| $P_1$ | 0 | 7 |
| $P_2$ | 2 | 4 |
| $P_3$ | 4 | 1 |
| $P_4$ | 5 | 4 |

For SJF, our schedule is (arrival on top):



## SJF Chooses the Shortest Job Available at Each Decision

1. At time 0, only $P_1$ is ready, so it runs to time 7
2. At time 7, $P_3$ is shortest and runs for 1 unit
3. $P_2$ and $P_4$ then run for 4 units each

Waiting times are $0, 6, 3, 7$, so:

$$\text{average waiting time} = \frac{0 + 6 + 3 + 7}{4} = 4$$

## SJF Needs Predictions and Can Starve Long Jobs

With exact burst lengths, non-preemptive SJF minimizes average waiting time.

Real systems do not know the future:

- They can estimate the next burst from earlier behavior
- A poor estimate may choose the wrong process
- Repeated short arrivals can postpone a long job indefinitely

The policy improves an average but does not guarantee fairness.

## SRTF Preempts When a Shorter Remaining Job Arrives

Shortest Remaining Time First is the preemptive form of SJF.

At each arrival or scheduling event:

- compare the remaining times of all ready processes,
- run the process with the least work left, and
- preempt the current process if a shorter one becomes ready.

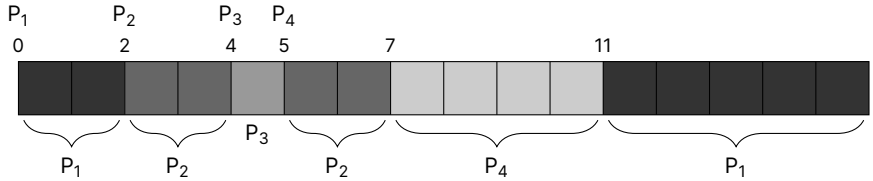
Exact remaining times are still an idealized assumption.

# Trace SRTF as Shorter Jobs Arrive

Consider the same processes and arrival times as SJF:

| Process | Arrival Time | Burst Time |
|---|---|---|
| $P_1$ | 0 | 7 |
| $P_2$ | 2 | 4 |
| $P_3$ | 4 | 1 |
| $P_4$ | 5 | 4 |

For SRTF, our schedule is (arrival on top):



## Preemption Reduces the Average Wait to Three Units

- $P_2$ preempts $P_1$ at time 2
- $P_3$ preempts $P_2$ at time 4
- After $P_3$ finishes, the shortest remaining job continues

Waiting times are $9, 1, 0, 2$:

$$\text{average waiting time} = \frac{9 + 1 + 0 + 2}{4} = 3$$

The lower average comes from letting short arrivals pass longer work.

## Round-Robin Shares the CPU with a Fixed Quantum

Round-Robin keeps a FIFO ready queue like FCFS, but limits each turn.

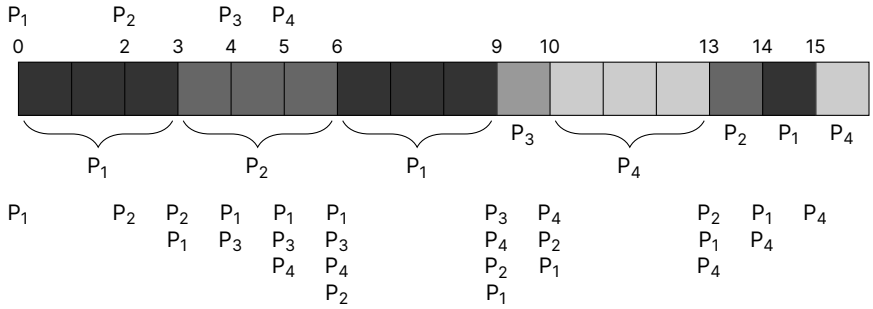
1. Run the process at the front for at most one quantum
2. If it finishes or blocks, choose the next ready process
3. If the quantum expires, preempt it and place it at the back

A shorter quantum improves response time but creates more context switches.

# Trace RR with a Three-Unit Quantum

| Process | Arrival Time | Burst Time |
|---|---|---|
| $P_1$ | 0 | 7 |
| $P_2$ | 2 | 4 |
| $P_3$ | 4 | 1 |
| $P_4$ | 5 | 4 |

For RR, our schedule is (arrival on top, queue on bottom):



# A Three-Unit Quantum Balances Response and Switching

Number of context switches: 7

Average waiting time: $\frac{8+8+5+7}{4} = 7$

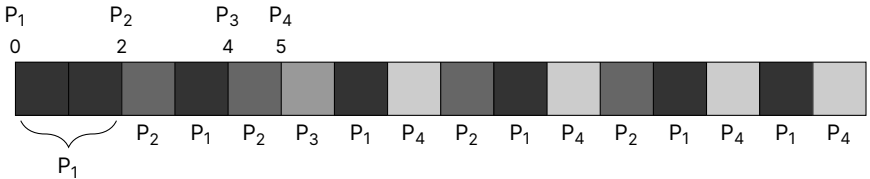
Average response time: $\frac{0+1+5+5}{4} = 2.75$

Note: on ties (a new process arrives while one is preempted), favor the new one

# Trace RR with a One-Unit Quantum

| Process | Arrival Time | Burst Time |
|---|---|---|
| $P_1$ | 0 | 7 |
| $P_2$ | 2 | 4 |
| $P_3$ | 4 | 1 |
| $P_4$ | 5 | 4 |

For RR, our schedule is (arrival on top, queue on bottom):




| | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $P_1$ | $P_1$ | $P_2$ | $P_1$ | $P_2$ | $P_3$ | $P_1$ | $P_4$ | $P_2$ | $P_1$ | $P_4$ | $P_2$ | $P_1$ | $P_4$ | $P_1$ | $P_4$ |
| | | $P_1$ | $P_2$ | $P_3$ | $P_1$ | $P_4$ | $P_2$ | $P_1$ | $P_4$ | $P_2$ | $P_1$ | $P_4$ | $P_1$ | $P_4$ | |
| | | | | $P_1$ | $P_4$ | $P_2$ | $P_1$ | $P_4$ | $P_2$ | $P_1$ | $P_4$ | | | | |
| | | | | | $P_2$ | | | | | | | | | | |

# A One-Unit Quantum Improves Response but Switches Often

Number of context switches: 14

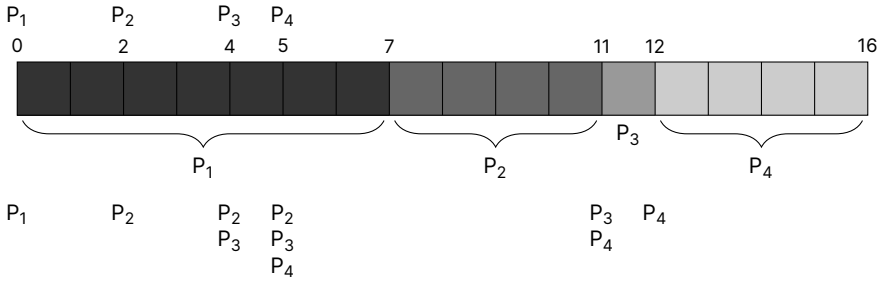
Average waiting time: $\frac{8+6+1+7}{4} = 5.5$

Average response time: $\frac{0+0+1+2}{4} = 0.75$

# Trace RR with a Ten-Unit Quantum

| Process | Arrival Time | Burst Time |
|---|---|---|
| $P_1$ | 0 | 7 |
| $P_2$ | 2 | 4 |
| $P_3$ | 4 | 1 |
| $P_4$ | 5 | 4 |

For RR, our schedule is (arrival on top, queue on bottom):



## A Ten-Unit Quantum Behaves Like FCFS Here

Number of context switches: 3

Average waiting time: $\frac{0+5+7+7}{4} = 4.75$

Average response time: $\frac{0+5+7+7}{4} = 4.75$

Note: in this case it's the same as FCFS without preemptions

## The Quantum Controls RR's Main Trade-Off

| **Quantum** | Benefit | Cost |
|---|---|---|
| **Short** | Fast first response and frequent turns | More context-switch overhead |
| **Medium** | Balances responsiveness and overhead | Depends on workload |
| **Long** | Few context switches | Approaches FCFS and delays response |

RR provides regular opportunities to run, but it does not minimize every job's waiting or turnaround time.

## Summary: Scheduling Requires Both Control and Trade-Offs

- Traps and interrupts return control from a process to the kernel
- A context switch saves one execution state and restores another
- Timer interrupts make preemptive scheduling possible
- FCFS is simple but sensitive to arrival order
- SJF and SRTF reduce average waiting when burst estimates are accurate
- RR improves fairness and response through bounded time quanta
- No policy optimizes waiting, response, throughput, overhead, and fairness at once

**Key distinction:** mechanism determines how the CPU switches; policy determines which ready process receives it.